Tema:

Investigacion Unidad 2 de Programación en Ambiente Cliente/Servidor

Docente:

Jenner Noel Che Méndez

Carrera:

Ingeniería en informática

Alumnos:

José Abimael Cisneros Escamilla

Semestre:

7AI

Fecha:

03 de Noviembre de 2026

Unidad 2. API HTTP/REST y persistencia de datos

Indice

Introducción 3

2.1. Recursos, URI, métodos HTTP, encabezados, códigos de estado, JSON, idempotencia y restricciones REST 4

2.2. Diseño API-first con OpenAPI 3.1 o superior compatible: esquemas, parámetros, respuestas, ejemplos y versionado 8

2.3. Validación de entrada, manejo uniforme de errores, serialización, paginación, filtrado, ordenamiento y búsqueda 11

2.5. Separación por capas: rutas/controladores, casos de uso o servicios, dominio y acceso a datos 17

2.6. Consumo desde HTML/Bootstrap/JavaScript o Flutter/Dart; estados de carga, error, vacío y éxito 19

2.7. Pruebas de endpoints y colección reproducible de solicitudes 21

Conclusión 23

Fuentes Bibliograficas 24

Introducción

En esta investigación se analizarán los principales conceptos y procedimientos que forman parte del desarrollo de interfaces de programación de aplicaciones (API), las cuales permiten la comunicación y el intercambio de información entre diferentes sistemas, aplicaciones y servicios. Actualmente, las API son herramientas fundamentales para el desarrollo de software, ya que facilitan la conexión entre aplicaciones web, dispositivos móviles y bases de datos.

Durante esta unidad se estudiarán los principios de la arquitectura REST, el funcionamiento de los recursos y las URI, los métodos HTTP, los códigos de estado y el formato JSON. También se abordará el diseño de API mediante el enfoque API-first y la especificación OpenAPI 3.1, además de los mecanismos de validación, manejo de errores, paginación y búsqueda de información.

Asimismo, se explicará cómo conectar una API con sistemas gestores de bases de datos como MySQL, MariaDB y PostgreSQL, la importancia de organizar el código mediante diferentes capas y las formas de consumir los servicios desde aplicaciones desarrolladas con HTML, Bootstrap, JavaScript o Flutter. Finalmente, se revisarán las pruebas de endpoints y la creación de colecciones de solicitudes que permitan comprobar y reproducir el funcionamiento de una API.

El propósito de estos conocimientos es comprender cómo diseñar, desarrollar, organizar y probar una API que sea segura, eficiente, fácil de mantener y compatible con diferentes aplicaciones.

2.1. Recursos, URI, métodos HTTP, encabezados, códigos de estado, JSON, idempotencia y restricciones REST

Una API es una interfaz que permite que diferentes aplicaciones se comuniquen entre sí mediante un conjunto de reglas y procedimientos establecidos. En el desarrollo de servicios web, uno de los estilos arquitectónicos más utilizados es REST, que significa Transferencia de Estado Representacional. Este estilo permite organizar la comunicación alrededor de recursos que pueden consultarse, crearse, modificarse o eliminarse.

Recursos y URI

Un recurso representa cualquier elemento o información que una API pone a disposición de sus usuarios. Algunos ejemplos pueden ser alumnos, profesores, préstamos, productos, usuarios o reservaciones.

Cada recurso se identifica mediante una URI (Identificador Uniforme de Recursos), que permite localizarlo dentro de una API. Por ejemplo:

- https://ejemplo.com/api/usuarios: representa la colección de usuarios.
- https://ejemplo.com/api/usuarios/15: representa al usuario cuyo identificador es 15.
- https://ejemplo.com/api/prestamos/8: representa un préstamo específico.

Las URI deben tener nombres claros y utilizar preferentemente sustantivos para identificar los recursos, evitando incluir acciones como crear, modificar o eliminar en sus direcciones. Las acciones se indican mediante los métodos HTTP.

Es importante señalar que una URL es un tipo de URI que identifica un recurso mediante su ubicación. Por lo tanto, aunque ambos términos están relacionados, no significan exactamente lo mismo (Mozilla, s. f.-a).

Métodos HTTP

Los métodos HTTP indican qué operación desea realizar el cliente sobre un recurso. Los más utilizados en una API REST son:

- **GET:** permite consultar u obtener información de un recurso.
- **POST:** permite enviar información al servidor y normalmente crear un nuevo recurso.
- **PUT:** permite reemplazar completamente la representación de un recurso.
- **PATCH:** permite modificar parcialmente un recurso.
- **DELETE:** permite eliminar un recurso.
- **HEAD:** obtiene información de los encabezados de una respuesta sin solicitar su contenido.

- **OPTIONS:** permite conocer los métodos y opciones de comunicación disponibles para un recurso.

Por ejemplo, en una API de préstamos, el método GET puede consultar los préstamos registrados, POST puede registrar uno nuevo, PUT puede reemplazar sus datos, PATCH puede modificar solamente su estado y DELETE puede eliminar un registro cuando las reglas del sistema lo permitan (Mozilla, s. f.-b).

Encabezados HTTP

Los encabezados HTTP son elementos que acompañan las solicitudes y respuestas para proporcionar información adicional sobre la comunicación entre el cliente y el servidor.

Entre los encabezados más importantes se encuentran:

- Content-Type: indica el formato del contenido que se está enviando. Por ejemplo, application/json.
- Accept: indica qué tipo de respuesta puede recibir el cliente.
- Authorization: permite enviar credenciales o tokens para identificar y autorizar al usuario.
- Cache-Control: establece instrucciones relacionadas con el almacenamiento de respuestas en caché.
- Location: puede indicar la dirección de un recurso que acaba de crearse o al que se debe acceder.

Estos encabezados permiten que el servidor interprete correctamente las solicitudes y genere respuestas compatibles con las necesidades del cliente.

Códigos de estado HTTP

Los códigos de estado HTTP informan al cliente sobre el resultado de una solicitud. Se dividen en cinco categorías:

Categoría	Significado
1xx	Respuestas informativas.
2xx	Solicitudes procesadas correctamente.
3xx	Redirecciones.
4xx	Errores relacionados con la solicitud del cliente.
5xx	Errores internos del servidor.

Algunos códigos utilizados frecuentemente en el desarrollo de API son:

- **200 OK:** la solicitud se realizó correctamente.
- **201 Created:** se creó un nuevo recurso.
- **204 No Content:** la operación fue exitosa, pero no devuelve contenido.
- **400 Bad Request:** la solicitud contiene datos o una estructura incorrecta.
- **401 Unauthorized:** se requiere autenticación.
- **403 Forbidden:** el usuario no tiene permiso para realizar la operación.
- **404 Not Found:** el recurso solicitado no existe.
- **409 Conflict:** existe un conflicto con el estado actual del recurso.
- **422 Unprocessable Content:** los datos recibidos no cumplen las condiciones de validación.
- **500 Internal Server Error:** ocurrió un error inesperado en el servidor.

El uso correcto de estos códigos facilita la identificación de problemas y permite que las aplicaciones consumidoras respondan adecuadamente (Mozilla, s. f.-c).

Formato JSON

JSON (JavaScript Object Notation) es un formato de intercambio de información que permite representar datos estructurados mediante pares de claves y valores. Es ampliamente utilizado por las API debido a su facilidad de lectura y compatibilidad con distintos lenguajes de programación.

Un ejemplo de respuesta JSON sería:


```
{
  "id": 15,
  "nombre": "Carlos",
  "correo": "carlos@ejemplo.com",
  "estado": "activo"
}
```


Los objetos JSON están formados por propiedades y valores. También pueden contener arreglos, números, cadenas de texto, valores booleanos y otros objetos.

La serialización consiste en convertir los datos de un programa a un formato que pueda transmitirse, mientras que la deserialización permite recuperar esos datos y convertirlos nuevamente en estructuras que pueda utilizar una aplicación (Mozilla, s. f.-d).

Idempotencia

La idempotencia es una propiedad que establece que repetir una misma operación varias veces debe producir el mismo efecto final que ejecutarla una sola vez.

Por ejemplo, si se realiza una solicitud PUT para establecer el estado de un préstamo como "entregado", repetir esa misma solicitud no debería generar cambios adicionales después de la primera ejecución.

Entre los métodos HTTP, GET, PUT y DELETE se consideran idempotentes. En cambio, POST no garantiza esta propiedad, debido a que repetir una solicitud para crear un registro puede generar varios registros nuevos.

Esta característica es importante para evitar modificaciones duplicadas cuando ocurren problemas de conexión o cuando un cliente necesita volver a enviar una solicitud (Microsoft, s. f.-a).

Restricciones de REST

REST establece una serie de restricciones arquitectónicas que permiten organizar la comunicación entre clientes y servidores:

1. Cliente-servidor: separa las responsabilidades de la aplicación que solicita información y el servidor que la proporciona.
2. Sin estado: cada solicitud debe contener la información necesaria para ser procesada, sin depender de una sesión almacenada entre solicitudes.
3. Caché: permite almacenar temporalmente determinadas respuestas para evitar solicitudes innecesarias.
4. Interfaz uniforme: establece una forma consistente de identificar y manipular los recursos.
5. Sistema en capas: permite utilizar intermediarios como servidores proxy, balanceadores de carga o sistemas de seguridad sin que el cliente tenga que conocer su funcionamiento interno.
6. Código bajo demanda: permite que el servidor envíe código ejecutable al cliente para ampliar temporalmente sus funcionalidades. Es una restricción opcional.

2.2. Diseño API-first con OpenAPI 3.1 o superior compatible: esquemas, parámetros, respuestas, ejemplos y versionado

El enfoque API-first consiste en diseñar primero la interfaz y el contrato de una API antes de comenzar con su implementación. Esto significa que se establece cómo funcionará el servicio, qué recursos tendrá disponibles, qué información recibirá y qué respuestas proporcionará.

Este método permite que los integrantes de un equipo conozcan las características de la API antes de desarrollar sus diferentes componentes. Por ejemplo, una persona puede trabajar en el backend mientras otra desarrolla una aplicación web que utilizará los endpoints definidos previamente.

OpenAPI 3.1

OpenAPI es una especificación utilizada para describir las características y operaciones de una API HTTP mediante un documento estructurado. Su finalidad es establecer un contrato que pueda ser comprendido tanto por desarrolladores como por herramientas de software.

La versión 3.1 permite describir los endpoints, métodos HTTP, parámetros, esquemas de datos, solicitudes, respuestas y mecanismos de autenticación. También utiliza JSON Schema para representar la estructura de los datos que intercambia una API (OpenAPI Initiative, 2025).

Entre los elementos principales de un documento OpenAPI se encuentran:

- **openapi:** indica la versión de la especificación utilizada.
- **info:** contiene el nombre, descripción y versión de la API.
- **servers:** establece las direcciones de los servidores disponibles.
- **paths:** define los endpoints y sus operaciones.
- **parameters:** describe los parámetros que recibe una solicitud.
- **requestBody:** establece la estructura de los datos enviados.
- **responses:** define las respuestas que puede generar cada operación.
- **components/schemas:** permite reutilizar esquemas de datos en diferentes endpoints.
- **security:** describe los mecanismos de autenticación y autorización.

Por ejemplo, un sistema de préstamos puede definir el endpoint /api/v1/prestamos, el cual utiliza el método POST para registrar un préstamo. El contrato especificaría los campos requeridos, sus tipos de datos, el código de respuesta esperado y un ejemplo del contenido JSON.

Esquemas

Los esquemas describen cómo deben estar organizados los datos. En ellos se establecen los nombres de las propiedades, sus tipos de datos y las restricciones correspondientes.

Por ejemplo, un esquema de préstamo podría contener:

- id: número entero que identifica el préstamo.
- usuario_id: número entero que identifica al usuario.
- fecha: fecha en la que se realiza el préstamo.
- estado: texto que representa el estado actual.
- observaciones: campo de texto opcional.

Los esquemas ayudan a mantener una estructura uniforme y permiten validar la información que entra y sale de una API.

Parámetros

Los parámetros permiten que el cliente envíe información adicional para controlar una operación. Los más comunes son:

- Parámetros de ruta: forman parte de la dirección, como /prestamos/12.
- Parámetros de consulta: aparecen después del símbolo ?, como /prestamos?estado=pendiente.
- Parámetros de encabezado: se envían mediante los encabezados HTTP.
- Parámetros de cookies: permiten transmitir información mediante cookies cuando el diseño del servicio lo requiere.

Respuestas y ejemplos

En OpenAPI se documentan las posibles respuestas de cada operación. Una solicitud puede devolver un código 200 cuando la consulta es exitosa, 201 cuando se crea un recurso o 404 cuando el elemento solicitado no existe.

También se pueden incluir ejemplos de solicitudes y respuestas JSON para mostrar claramente cómo debe utilizarse el servicio.

Versionado

El versionado permite realizar modificaciones y agregar funcionalidades sin afectar necesariamente a las aplicaciones que utilizan versiones anteriores de la API.

Una forma común de implementarlo es agregar el número de versión a la ruta:

- /api/v1/prestamos
- /api/v2/prestamos

Cuando se realizan cambios incompatibles, se puede desarrollar una nueva versión y mantener temporalmente disponible la anterior. Esto permite actualizar los clientes de manera organizada y evitar interrupciones en los servicios existentes.

2.3. Validación de entrada, manejo uniforme de errores, serialización, paginación, filtrado, ordenamiento y búsqueda

Una API debe comprobar que los datos recibidos sean correctos antes de procesarlos. También necesita manejar los errores de manera organizada y ofrecer mecanismos que faciliten la consulta de grandes cantidades de información.

Validación de entrada

La validación de entrada es el proceso mediante el cual se revisa que la información enviada por el cliente cumpla con las condiciones establecidas por el sistema.

Entre las validaciones más comunes se encuentran:

- Campos obligatorios: verificar que no falten datos necesarios.
- Tipos de datos: comprobar que un campo numérico no reciba texto cuando no está permitido.
- Longitud: establecer límites mínimos o máximos para los textos.
- Formatos: verificar que fechas, correos electrónicos y otros valores tengan una estructura válida.
- Rangos: comprobar que los números se encuentren dentro de los límites permitidos.
- Reglas de negocio: verificar que la operación solicitada tenga sentido dentro del funcionamiento del sistema.

Por ejemplo, en una API de préstamos no debería permitirse registrar un préstamo con un usuario inexistente o establecer una fecha de devolución anterior a la fecha de entrega.

La validación permite detectar errores antes de que lleguen a la base de datos y reduce la posibilidad de almacenar información incorrecta. Frameworks como FastAPI utilizan modelos de datos y validaciones para comprobar automáticamente diferentes tipos de entrada (FastAPI, s. f.-b).

Manejo uniforme de errores

El manejo de errores consiste en identificar situaciones en las que una solicitud no puede procesarse correctamente y responder de una manera clara.

Para mantener la uniformidad, una API debe utilizar una estructura consistente para informar los problemas. Por ejemplo:

```
{
  "error": {
    "codigo": "VALIDACION",
```

```
  "mensaje": "Los datos enviados son incorrectos.",
  "detalles": [
   {
    "campo": "usuario_id",
    "mensaje": "El usuario no existe."
   }
  ]
 }
}
```


Este formato permite que las aplicaciones puedan identificar el tipo de error y mostrar mensajes comprensibles a sus usuarios.

También es importante utilizar códigos HTTP adecuados, separar los errores del cliente de los errores internos del servidor y evitar mostrar información confidencial, contraseñas o detalles internos del sistema.

FastAPI permite utilizar excepciones HTTP y manejadores personalizados para controlar las respuestas de error de una aplicación (FastAPI, s. f.-c).

Serialización

La serialización permite convertir los datos de una aplicación a un formato que pueda transmitirse mediante una API, generalmente JSON.

Por ejemplo, un objeto que contiene los datos de un usuario puede convertirse en una estructura JSON para enviarse al navegador. Después, el cliente puede deserializar esa información y utilizarla para mostrar los datos en pantalla.

Este proceso facilita la comunicación entre aplicaciones que utilizan lenguajes de programación diferentes.

Paginación

La paginación consiste en dividir grandes conjuntos de resultados en grupos más pequeños. En lugar de enviar todos los registros disponibles, la API entrega solamente una cantidad determinada por solicitud.

Por ejemplo:

GET /api/prestamos?limit=10&offset=0

En este caso, limit=10 establece que se desean obtener hasta diez registros y offset=0 indica desde qué posición comienza la consulta.

La paginación mejora el rendimiento y evita que el servidor y el cliente tengan que procesar demasiada información al mismo tiempo.

Filtrado

El filtrado permite consultar solamente los registros que cumplen determinadas condiciones.

Por ejemplo:

GET /api/prestamos?estado=pendiente

Esta solicitud obtiene los préstamos que tienen el estado pendiente.

También se pueden utilizar varios filtros para buscar información por fecha, usuario, categoría o cualquier otra propiedad permitida por el sistema.

Ordenamiento

El ordenamiento permite establecer el orden en el que se presentan los resultados.

Por ejemplo:

GET /api/prestamos?sort=fecha&order=desc

Esta solicitud puede organizar los registros por fecha, mostrando primero los más recientes.

Es importante que la API permita ordenar solamente por campos autorizados para evitar consultas incorrectas o accesos no permitidos.

Búsqueda

La búsqueda permite localizar información mediante palabras o criterios determinados.

Por ejemplo:

GET /api/usuarios?search=carlos

El servidor puede utilizar el valor recibido para localizar coincidencias en campos como nombre, apellidos o correo electrónico.

2.4. Acceso a MySQL/MariaDB o PostgreSQL/Supabase: repositorios, transacciones, migraciones, restricciones e integridad

Las API generalmente necesitan almacenar y consultar información de manera permanente. Para conseguirlo, pueden conectarse a diferentes sistemas gestores de bases de datos, como MySQL, MariaDB o PostgreSQL.

Estos sistemas permiten organizar la información en tablas relacionadas y realizar operaciones para insertar, consultar, modificar y eliminar registros.

Conexión con bases de datos

Para establecer la comunicación entre una API y una base de datos se necesita configurar la conexión correspondiente. Generalmente se requieren datos como el servidor, puerto, nombre de la base de datos, usuario y contraseña.

La conexión debe administrarse mediante herramientas o bibliotecas que permitan ejecutar consultas y controlar las operaciones.

FastAPI puede utilizar distintas soluciones de acceso a datos, como SQLAlchemy y SQLModel, compatibles con gestores como MySQL, PostgreSQL y SQLite (FastAPI, s. f.-e).

Patrón repositorio

El patrón repositorio consiste en organizar el acceso a los datos mediante componentes encargados de realizar las operaciones sobre la base de datos.

En lugar de escribir todas las consultas directamente en las rutas de la API, se crean repositorios responsables de consultar, insertar, actualizar o eliminar información.

Por ejemplo, un repositorio de préstamos podría incluir operaciones como:

- Obtener todos los préstamos.
- Buscar un préstamo mediante su identificador.
- Registrar un préstamo.
- Actualizar el estado de un préstamo.
- Eliminar un préstamo.

De esta manera, la lógica del sistema no depende directamente de las consultas SQL y resulta más sencillo modificar el funcionamiento interno de la base de datos.

Transacciones

Una transacción es un conjunto de operaciones que se ejecutan como una unidad. Su finalidad es mantener la consistencia de los datos y evitar que una operación quede incompleta.

Las transacciones se relacionan con las propiedades ACID:

- Atomicidad: todas las operaciones se realizan o ninguna queda aplicada.
- Consistencia: los datos deben mantener las reglas de integridad establecidas.
- Aislamiento: las transacciones concurrentes deben evitar interferencias incorrectas.
- Durabilidad: los cambios confirmados deben permanecer almacenados.

Para administrar una transacción se utilizan principalmente dos instrucciones:

- COMMIT: confirma los cambios realizados.
- ROLLBACK: deshace los cambios que todavía no se han confirmado.

Por ejemplo, al registrar un préstamo, el sistema podría necesitar crear el registro del préstamo y actualizar la disponibilidad del recurso. Si una de estas operaciones falla, la transacción permite evitar que solamente una parte de los cambios quede guardada (Tutorial MySQL, s. f.).

Migraciones

Las migraciones son procedimientos que permiten modificar la estructura de una base de datos de manera controlada.

Se utilizan cuando es necesario agregar tablas, modificar columnas, crear relaciones o cambiar restricciones sin tener que reconstruir manualmente toda la base de datos.

Por ejemplo, si un sistema necesita agregar el campo fecha_devolucion a la tabla de préstamos, se puede crear una migración que registre ese cambio.

Las migraciones también permiten llevar un historial de las modificaciones realizadas y mantener sincronizada la estructura de la base de datos entre diferentes entornos de desarrollo y producción.

Antes de aplicar una migración es importante revisar los cambios y probarlos, debido a que determinadas modificaciones pueden ocasionar pérdida de información (Microsoft, s. f.-d).

Restricciones e integridad de datos

Las restricciones son condiciones que se aplican a las tablas para evitar que se almacene información incorrecta.

Entre las principales se encuentran:

- PRIMARY KEY: identifica de manera única cada registro.
- FOREIGN KEY: establece relaciones entre tablas y ayuda a mantener la integridad referencial.
- NOT NULL: impide que un campo obligatorio quede vacío.
- UNIQUE: evita que determinados valores se repitan.

- CHECK: permite establecer condiciones que deben cumplir los valores de una columna.

La integridad de datos consiste en mantener la exactitud, validez y coherencia de la información almacenada.

2.5. Separación por capas: rutas/controladores, casos de uso o servicios, dominio y acceso a datos

La separación por capas es una forma de organizar el código de una aplicación mediante diferentes componentes, cada uno con responsabilidades específicas.

Su principal objetivo es evitar que todo el funcionamiento de un sistema se concentre en un solo archivo o componente. Esto facilita la comprensión, el mantenimiento, las pruebas y la ampliación del proyecto.

Una API puede organizarse mediante las siguientes capas:

Rutas o controladores

Son los puntos de entrada de las solicitudes HTTP. Se encargan de recibir las peticiones, identificar el método utilizado, obtener los parámetros, comprobar la información de entrada y devolver las respuestas correspondientes.

Por ejemplo, una ruta puede recibir una solicitud POST para registrar un préstamo y enviar los datos al servicio encargado de procesarla.

Los controladores deben evitar concentrar toda la lógica del sistema. Su función principal es gestionar la comunicación entre el cliente y las demás capas.

Casos de uso o servicios

Esta capa contiene la lógica que determina cómo debe funcionar el sistema. Se encarga de coordinar las operaciones necesarias para cumplir una acción solicitada.

Por ejemplo, un servicio para registrar préstamos puede comprobar que el usuario exista, verificar la disponibilidad del recurso y solicitar al repositorio que almacene la operación.

Los servicios representan las reglas y procesos que permiten que la aplicación realice sus funciones correctamente.

Dominio

El dominio representa los conceptos y reglas principales del sistema. Contiene elementos como entidades, atributos, relaciones y condiciones que determinan el comportamiento de la aplicación.

Por ejemplo, en un sistema de préstamos pueden existir entidades como Usuario, Recurso y Prestamo, cada una con sus características y reglas correspondientes.

Esta capa permite separar las reglas del negocio de los detalles técnicos relacionados con HTTP o con el gestor de base de datos.

Acceso a datos

Esta capa se encarga de comunicarse directamente con la base de datos. Incluye los repositorios, las consultas SQL, las conexiones y las operaciones de persistencia.

Su responsabilidad es recuperar, almacenar, modificar o eliminar información según las solicitudes de los servicios.

Importancia de la separación por capas

La organización por capas ofrece diferentes beneficios:

- Facilita encontrar y corregir errores.
- Permite modificar una parte del sistema sin afectar innecesariamente a las demás.
- Evita duplicar código.
- Facilita realizar pruebas independientes.
- Permite cambiar la tecnología de almacenamiento con menos modificaciones en la lógica del sistema.

2.6. Consumo desde HTML/Bootstrap/JavaScript o Flutter/Dart; estados de carga, error, vacío y éxito

El consumo de una API consiste en utilizar sus endpoints desde otra aplicación para enviar solicitudes, recibir información y mostrar los resultados al usuario.

Las API pueden consumirse desde diferentes tecnologías, como aplicaciones web desarrolladas con HTML, Bootstrap y JavaScript, o aplicaciones móviles construidas con Flutter y Dart.

Consumo desde HTML, Bootstrap y JavaScript

HTML permite organizar los elementos de una página web, Bootstrap facilita el diseño visual mediante componentes y estilos, mientras que JavaScript se encarga de realizar las solicitudes y procesar las respuestas de la API.

Para comunicarse con el servidor se puede utilizar la función fetch(), que permite realizar solicitudes HTTP de manera asíncrona.

Es importante comprobar response.ok, porque la función fetch no considera automáticamente que una respuesta HTTP 404 o 500 sea un error de conexión. Por ello, el programador debe verificar el código de estado recibido.

Consumo desde Flutter y Dart

Flutter permite desarrollar aplicaciones para diferentes plataformas utilizando el lenguaje de programación Dart.

Una aplicación Flutter puede comunicarse con una API mediante solicitudes HTTP utilizando paquetes como http.

El procedimiento general consiste en:

1. Configurar la dirección del servicio.
2. Realizar una solicitud HTTP.
3. Recibir la respuesta del servidor.
4. Verificar el código de estado.
5. Convertir la información JSON en objetos o estructuras de Dart.
6. Presentar los resultados en la interfaz de la aplicación.

Por ejemplo, una aplicación móvil puede consultar los préstamos registrados en una base de datos y mostrarlos en una lista.

Tanto JavaScript como Flutter permiten consumir una misma API, siempre que el servicio proporcione los endpoints y formatos necesarios para ambos clientes.

Estados de la interfaz

Cuando una aplicación consume una API, es importante informar al usuario sobre lo que ocurre durante la comunicación.

Se pueden identificar cuatro estados principales:

- **Carga:** indica que la aplicación está esperando la respuesta de la API. Puede mostrarse un indicador giratorio o un mensaje como "Cargando información".
- **Error:** aparece cuando la solicitud falla, el servidor no responde o se presenta un problema al procesar la información. Se puede mostrar un mensaje y permitir que el usuario intente nuevamente.
- **Vacío:** indica que la solicitud fue exitosa, pero no existen registros para mostrar. Por ejemplo, cuando no hay préstamos registrados.
- **Éxito:** aparece cuando la API responde correctamente y proporciona información que puede mostrarse en la interfaz.

La implementación de estos estados mejora la experiencia del usuario porque evita que la aplicación parezca bloqueada o que se confunda una lista sin resultados con un error.

2.7. Pruebas de endpoints y colección reproducible de solicitudes

Las pruebas de endpoints permiten comprobar que cada operación de una API funcione correctamente. Su propósito es verificar que las solicitudes sean procesadas de acuerdo con lo establecido en el contrato y que las respuestas contengan los datos y códigos de estado esperados.

Un endpoint es una dirección específica de una API mediante la cual un cliente puede realizar una operación.

Por ejemplo:

GET http://127.0.0.1:8000/api/prestamos

Este endpoint puede utilizarse para consultar los préstamos registrados.

Tipos de pruebas

Al desarrollar una API se deben realizar diferentes pruebas:

- Pruebas de consulta: comprueban que los endpoints GET devuelvan la información correcta.
- Pruebas de creación: verifican que POST registre nuevos recursos.
- Pruebas de actualización: comprueban que PUT y PATCH modifiquen la información correctamente.
- Pruebas de eliminación: verifican que DELETE realice la operación correspondiente.
- Pruebas de validación: comprueban que los datos incorrectos sean rechazados.
- Pruebas de errores: verifican que la API responda adecuadamente cuando un recurso no existe o ocurre un problema.
- Pruebas de seguridad: comprueban que las operaciones protegidas requieran los permisos correspondientes.

También es importante probar diferentes situaciones, como enviar campos vacíos, identificadores inexistentes, valores incorrectos o solicitudes repetidas.

Herramientas para realizar pruebas

Existen diferentes herramientas que facilitan la comprobación de los endpoints. Una de las más conocidas es Postman, que permite realizar solicitudes HTTP, configurar encabezados, enviar cuerpos JSON y revisar las respuestas del servidor.

También permite organizar las solicitudes en colecciones para mantenerlas agrupadas y facilitar su reutilización.

Por ejemplo, una colección para una API de préstamos podría contener las siguientes solicitudes:

1. Consultar todos los préstamos.
2. Consultar un préstamo por su identificador.
3. Registrar un nuevo préstamo.
4. Actualizar la información de un préstamo.
5. Eliminar un préstamo.
6. Intentar registrar información incorrecta para comprobar las validaciones.

Colección reproducible de solicitudes

Una colección reproducible es un conjunto organizado de solicitudes que puede guardarse, exportarse y utilizarse posteriormente para volver a probar una API.

Para facilitar su reutilización, debe incluir:

- El método HTTP de cada solicitud.
- La dirección del endpoint.
- Los encabezados necesarios.
- Los parámetros utilizados.
- Los cuerpos JSON de ejemplo.
- Las respuestas o códigos de estado que se esperan.
- Las variables de entorno necesarias, como la dirección base del servidor.

Las direcciones y credenciales que cambian entre ambientes deben configurarse mediante variables. Además, no deben guardarse contraseñas ni tokens reales en una colección que se vaya a compartir.

Importancia de las pruebas

Las pruebas permiten detectar errores antes de que una API sea utilizada por otras aplicaciones. También ayudan a comprobar que los cambios realizados en el código no dañen operaciones que anteriormente funcionaban.

Frameworks como FastAPI permiten utilizar herramientas como pytest y TestClient para realizar solicitudes de prueba y comprobar automáticamente los códigos de estado y el contenido de las respuestas.

Conclusión

Para finalizar, el desarrollo de APIs requiere conocer diferentes conceptos y herramientas que trabajan en conjunto para permitir la comunicación entre aplicaciones y sistemas de información. A través de los principios REST, los métodos HTTP, las URI, los encabezados y el formato JSON es posible establecer una comunicación organizada y uniforme entre los clientes y los servidores.

También se comprendió la importancia de diseñar las API mediante un enfoque API-first y utilizar especificaciones como OpenAPI 3.1 para definir previamente sus operaciones, parámetros, esquemas y respuestas. De igual manera, la validación de entrada, el manejo de errores y los mecanismos de paginación, filtrado y búsqueda permiten construir servicios más confiables y eficientes.

Por otra parte, el acceso a bases de datos como MySQL, MariaDB y PostgreSQL facilita el almacenamiento y administración de la información, mientras que las transacciones, migraciones y restricciones ayudan a mantener su integridad. La separación del código mediante capas también representa una práctica importante, ya que permite organizar las responsabilidades y facilita el mantenimiento de los proyectos.

Finalmente, el consumo de API desde aplicaciones web y móviles, junto con las pruebas de endpoints y las colecciones reproducibles, permite comprobar que los servicios funcionen correctamente y que puedan utilizarse desde diferentes plataformas.

Fuentes Bibliograficas

- FastAPI. (s. f.-a). *Primeros pasos*. https://fastapi.tiangolo.com/es/tutorial/first-steps/
- https://fastapi.tiangolo.com/es/tutorial/sql-databases/
- FastAPI. (s. f.-f). *Pruebas*. https://fastapi.tiangolo.com/es/tutorial/testing/
- Google Cloud. (s. f.). *Conceptos básicos e implementación de la API de REST*. https://cloud.google.com/discover/what-is-rest-api?hl=es-419
- IBM. (s. f.). *¿Qué es el diseño de API?* https://www.ibm.com/mx-es/think/topics/api-design
- Mozilla. (s. f.-d). *Trabajar con JSON*. MDN Web Docs. https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting/JSON
- Mozilla. (s. f.-f). *Uso de Fetch*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- Mozilla. (s. f.-g). *Fetch API*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- OpenAPI Initiative. (2025, 19 de septiembre). *OpenAPI Specification v3.1.2*. https://spec.openapis.org/oas/v3.1
- Tutorial MySQL. (s. f.). *Transacciones en MySQL*. https://tutorialmysql.org/transacciones